

DBMS – PRODUCT REVIEW AND RATING MANAGEMENT SYSTEM

ZEPTO E-COMMERCE

1. DATABASE DESIGN:

REVIEW TABLE:

Attribute	Data Type	Description
REVIEW_ID	NUMBER	Unique ID for each review
CUSTOMER_ID	NUMBER	Identifies the customer
PRODUCT_ID	NUMBER	Identifies the product
REVIEW_TEXT	VARCHAR2(100)	Customer feedback
REVIEW_DATE	DATE	Date of the review

RATING TABLE:

Attribute	Data Type	Description
RATING_ID	NUMBER	Unique ID for each rating
REVIEW_ID	NUMBER	Identifies the related review
RATING_VALUE	NUMBER(1)	Rating given by the customer

2. TABLE RELATIONSHIPS:

- **CUSTOMER → REVIEW : One-to-Many**
- **PRODUCT → REVIEW : One-to-Many**
- **REVIEW → RATING : One-to-One**

3. TABLE CREATION:

CREATING REVIEW TABLE:

```
SQL> CREATE TABLE REVIEW(  
 2     REVIEW_ID NUMBER PRIMARY KEY,  
 3     CUSTOMER_ID NUMBER,  
 4     PRODUCT_ID NUMBER,  
 5     REVIEW_TEXT VARCHAR2(100),  
 6     REVIEW_DATE DATE,  
 7     FOREIGN KEY(CUSTOMER_ID) REFERENCES CUSTOMER(CUSTOMER_ID),  
 8     FOREIGN KEY(PRODUCT_ID) REFERENCES PRODUCT(PRODUCT_ID)  
 9 );
```

Table created.

CREATING RATING TABLE:

```
SQL> CREATE TABLE RATING(  
 2     RATING_ID NUMBER PRIMARY KEY,  
 3     REVIEW_ID NUMBER,  
 4     RATING_VALUE NUMBER(1),  
 5     FOREIGN KEY(REVIEW_ID) REFERENCES REVIEW(REVIEW_ID)  
 6 );
```

Table created.

4. DATA INSERTION:

REVIEW RECORDS:

```
SQL> INSERT INTO REVIEW  
 2 VALUES(701,101,101,'Fresh and good quality apple',  
 3 TO_DATE('25-09-2026','DD-MM-YYYY'));
```

1 row created.

```
SQL> INSERT INTO REVIEW  
 2 VALUES(702,102,102,'Bananas were fresh',  
 3 TO_DATE('26-09-2026','DD-MM-YYYY'));
```

1 row created.

```
SQL> INSERT INTO REVIEW  
 2 VALUES(703,103,103,'Tomatoes were good',  
 3 TO_DATE('27-09-2026','DD-MM-YYYY'));
```

1 row created.

```
SQL> INSERT INTO REVIEW  
 2 VALUES(704,104,104,'Milk was fresh',  
 3 TO_DATE('28-09-2026','DD-MM-YYYY'));
```

1 row created.

```
SQL> INSERT INTO REVIEW  
 2 VALUES(705,105,105,'Biscuits were tasty',  
 3 TO_DATE('29-09-2026','DD-MM-YYYY'));
```

1 row created.

RATING RECORDS:

```
SQL> INSERT INTO RATING
| 2 VALUES(801,701,5);

1 row created.

SQL> INSERT INTO RATING
| 2 VALUES(802,702,4);

1 row created.

SQL> INSERT INTO RATING
| 2 VALUES(803,703,3);

1 row created.

SQL> INSERT INTO RATING
| 2 VALUES(804,704,5);

1 row created.

SQL> INSERT INTO RATING
| 2 VALUES(805,705,4);

1 row created.
```

5. DISPLAY RECORDS:

REVIEW RECORDS:

```
SQL> SELECT * FROM REVIEW;

REVIEW_ID CUSTOMER_ID PRODUCT_ID
-----
REVIEW_TEXT
-----
REVIEW_DA
-----
| | | 701      101      101
Fresh and good quality apple
25-SEP-26
| | | 702      102      102
Bananas were fresh
26-SEP-26
| | | 703      103      103
Tomatoes were good
27-SEP-26
| | | 704      104      104
Milk was fresh
| | | 705      105      105
Biscuits were tasty
29-SEP-26
```

RATING RECORDS:

```
SQL> SELECT * FROM RATING;
```

RATING_ID	REVIEW_ID	RATING_VALUE
801	701	5
802	702	4
803	703	3
804	704	5
805	705	4

6. PRODUCT REVIEW DETAILS:

```
SQL> SELECT C.CUSTOMER_NAME,  
2         P.PRODUCT_NAME,  
3         R.REVIEW_TEXT,  
4         RT.RATING_VALUE,  
5         R.REVIEW_DATE  
6 FROM REVIEW R  
7 JOIN CUSTOMER C  
8 ON R.CUSTOMER_ID = C.CUSTOMER_ID  
9 JOIN PRODUCT P  
10 ON R.PRODUCT_ID = P.PRODUCT_ID  
11 JOIN RATING RT  
12 ON R.REVIEW_ID = RT.REVIEW_ID  
13 ORDER BY R.REVIEW_DATE;
```

CUSTOMER_NAME	PRODUCT_NAME
---------------	--------------

REVIEW_TEXT

RATING_VALUE	REVIEW_DA
--------------	-----------

AYESHA	APPLE
Fresh and good quality apple	
5	25-SEP-26

POOJA	BANANA
Bananas were fresh	
4	26-SEP-26

CUSTOMER_NAME	PRODUCT_NAME
---------------	--------------

REVIEW_TEXT

RATING_VALUE	REVIEW_DA
--------------	-----------

SHINY	TOMATO
Tomatoes were good	
3	27-SEP-26

RIYAVALLI	MILK
Milk was fresh	

CUSTOMER_NAME	PRODUCT_NAME
---------------	--------------

REVIEW_TEXT

RATING_VALUE	REVIEW_DA
--------------	-----------

5	28-SEP-26
---	-----------

AKSHAYA	BISCUIT
Biscuits were tasty	
4	29-SEP-26

7. AVERAGE PRODUCT RATINGS:

```
SQL> SELECT P.PRODUCT_NAME,  
2      AVG(RT.RATING_VALUE) AS AVERAGE_RATING  
3 FROM PRODUCT P  
4 JOIN REVIEW R  
5 ON P.PRODUCT_ID = R.PRODUCT_ID  
6 JOIN RATING RT  
7 ON R.REVIEW_ID = RT.REVIEW_ID  
8 GROUP BY P.PRODUCT_NAME  
9 ORDER BY P.PRODUCT_NAME;
```

PRODUCT_NAME	AVERAGE_RATING
APPLE	5
BANANA	4
BISCUIT	4
MILK	5
TOMATO	3

8. HIGHLY RATED PRODUCTS:

```
SQL> SELECT P.PRODUCT_NAME,  
2      AVG(RT.RATING_VALUE) AS AVERAGE_RATING  
3 FROM PRODUCT P  
4 JOIN REVIEW R  
5 ON P.PRODUCT_ID = R.PRODUCT_ID  
6 JOIN RATING RT  
7 ON R.REVIEW_ID = RT.REVIEW_ID  
8 GROUP BY P.PRODUCT_NAME  
9 HAVING AVG(RT.RATING_VALUE) >= 4  
10 ORDER BY AVERAGE_RATING DESC;
```

PRODUCT_NAME	AVERAGE_RATING
MILK	5
APPLE	5
BISCUIT	4
BANANA	4

9. RATING-WISE ANALYSIS:

```
SQL> SELECT RATING_VALUE,  
2      COUNT(*) AS TOTAL_REVIEWS  
3 FROM RATING  
4 GROUP BY RATING_VALUE  
5 ORDER BY RATING_VALUE DESC;
```

RATING_VALUE	TOTAL_REVIEWS
5	2
4	2
3	1